

O/L Examination Result Publishing System

Enterprise-ready technical and functional proposal

1. Project Overview

The proposed system is a secure, scalable, mobile-responsive web application for publishing and retrieving O/L Model Examination results.

The system will initially contain approximately 200-500 student records, but shall be designed to support 500 or more concurrent result-search requests, particularly during the high-traffic period immediately following result publication.

The system will primarily serve users located in Sri Lanka.

Students will be able to securely retrieve their examination results and view subject grades. Raw marks, total marks, averages, and rankings shall not be publicly displayed.

The application shall provide a separate administrative system for managing examinations, students, subjects, grades, imports, publication, and auditing.

2. Project Objectives

The system shall:

- Provide fast result retrieval during peak traffic.
- Support at least 500 concurrent result searches as an initial performance target.
- Protect student information from unauthorized access.
- Provide a simple mobile-first interface.
- Allow administrators to efficiently manage examination results.
- Support bulk Excel/CSV imports.
- Prevent accidental publication of incomplete results.
- Maintain an audit history of important administrative actions.
- Support future examinations without requiring database redesign.
- Provide backup and recovery procedures.
- Remain simple enough to operate using cost-effective cloud infrastructure.

3. System Users

3.1 Student / Public User

A student can:

- Visit the public result website.
- Enter the required result-search credentials.
- Retrieve their result after publication.
- View student information.
- View subject grades.
- View overall result status.
- Print the result.
- Download/print the result as PDF.

- Return to the result-search page.

Students do not require user accounts.

3.2 Administrator

The initial system shall support one authorized administrator.

The administrator can:

- Log in securely.
- Access the administration dashboard.
- Create and manage examinations.
- Add students.
- Edit student information.
- Delete students where permitted.
- Manage subjects.
- Enter and edit grades.
- Upload results using Excel/CSV.
- Validate imported data.
- Review incomplete results.
- Publish results.
- Unpublish results.
- Search and filter student records.
- Export appropriate result data.
- View publication status.
- View administrative audit logs.
- Log out securely.

4. High-Level System Architecture

The proposed architecture is:

```

STUDENTS / PARENTS
|
| HTTPS
|
v
VERCEL
CDN / Edge Network
|
+-----+-----+
| |
v v
Static Frontend Next.js Server
HTML/CSS/JS/Images API / Actions
|
Input Validation
|
Rate Limiting
|
v
Result Service
|
v
SUPABASE POSTGRESQL
+-----+-----+

```

```
| |
Indexed Data Backups
```

Administrative requests follow:

```
Administrator
|
v
/admin/login
|
v
Authentication
|
v
Secure Session
|
v
Protected Admin API
|
v
Supabase PostgreSQL
|
+---- Students
+---- Examinations
+---- Subjects
+---- Results
+---- Audit Logs
```

The architecture shall initially use a modular monolith rather than microservices.

5. Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Backend

- Next.js Server Actions and/or Route Handlers
- Server-side validation
- TypeScript

Database

- Supabase PostgreSQL

Database Security

- PostgreSQL constraints
- Database indexes
- Supabase Row Level Security where appropriate
- Server-side privileged database operations only
- Environment variables for private credentials

Deployment

- Vercel

Source Control

- Git

- GitHub

Excel/CSV Processing

- SheetJS or ExcelJS

PDF / Printing

Prefer:

- Browser print stylesheet

Optional:

- React PDF

The first version should avoid expensive server-side PDF generation unless required.

6. Scalability Requirements

The initial system shall support:

- Approximately 200-500 student records.
- 500+ concurrent result-search requests as the initial load-testing target.
- Future growth to additional examinations and student records.

The system workload is expected to be highly read-heavy after publication.

Peak traffic is expected immediately after results are released.

Therefore the architecture shall prioritize:

- Fast indexed database queries.
- Minimal database operations per result search.
- Small API responses.
- Efficient database connections.
- CDN delivery of static resources.
- Rate limiting.
- Lightweight frontend assets.

The system shall be load-tested before production result publication.

7. Performance Requirements

Performance targets under expected operating conditions:

- Public pages should load quickly on typical Sri Lankan mobile connections.
- Result lookup should normally complete within approximately 1 second.
- Static assets should be served through Vercel's edge/CDN infrastructure.
- Result queries must use indexed database columns.
- Database queries should return only required fields.
- Unnecessary frontend JavaScript should be minimized.

Performance testing should measure:

- Requests per second.
- Concurrent users.
- Average response time.

- p95 response time.
- p99 response time.
- HTTP error rate.
- Database response time.
- Failed requests.

Load tests should simulate:

```
100 concurrent users
250 concurrent users
500 concurrent users
750 concurrent users
1000 concurrent users
```

The required acceptance threshold should be established from the 500-user test.

8. Public Result Search

The public result page shall contain:

- Organization logo.
- Organization name.
- Examination name.
- Examination year.
- Search form.
- Important notices.
- Contact information.

Example:

UGSM O/L Model Examination Results 2026

9. Result Verification

Students shall be able to search for their examination results using either:

Index Number, OR NIC Number

The student shall not be required to provide both values.

Example using Index Number:

Index Number: OL2026001

OR using NIC Number:

NIC Number: 200312345678

10. Search Security

Search credentials shall be submitted using an HTTPS POST request.

Sensitive identifiers should not be included in query-string URLs.

Avoid:

```
/results?nic=200312345678
```

Prefer:

```
POST /api/results/search
```

The server shall:

1. Receive the request.

2. Normalize the input.
3. Validate the input.
4. Check rate limits.
5. Check publication status.
6. Search for the matching student.
7. Retrieve the student's grades.
8. Construct a sanitized response.
9. Return only permitted information.

11. Search Validation

The system shall:

- Reject empty fields.
- Remove leading/trailing spaces.
- Normalize Index Numbers.
- Validate NIC format.
- Reject malformed requests.
- Return only one matching student.
- Display a loading indicator during requests.
- Use generic error messages where appropriate to reduce information leakage.

Possible messages:

```
Please enter the required information.  
  
The provided information does not match an available result.  
  
Results have not been published yet.  
  
Too many attempts. Please try again shortly.  
  
Something went wrong. Please try again.
```

12. Rate Limiting and Anti-Abuse Protection

Public result search endpoints shall be rate limited.

Rate limiting shall help prevent:

- Automated Index Number enumeration.
- NIC enumeration.
- Brute-force result searches.
- Bots.
- Accidental request flooding.
- Basic denial-of-service attempts.

The system may apply additional verification or temporary blocking after suspicious activity.

Rate limits shall be tuned through load testing so legitimate students are not unnecessarily blocked during result publication.

13. Result Display

After successful verification, display:

- Student name.
- Index Number.
- Masked NIC Number.
- School.
- Examination name.
- Examination year.
- Examination center, where applicable.
- Subject grades.
- Overall result status.

Example:

```
Student Name: Mohamed Akeel  
Index Number: OL2026001  
NIC Number: *****5678  
School: Example College
```

The complete NIC shall never be displayed publicly.

14. Subject Results

Only grades shall be displayed.

Subject	Grade
Tamil	A
English	B
Mathematics	B
Science	C
History	A
Religion	A
ICT	B
Commerce	C

The system shall not display:

- Raw marks.
- Individual subject scores.
- Total marks.
- Average marks.
- Student ranking.

15. Supported Grades

Supported grades:

- A
- B
- C
- S

- W
- AB

Meaning:

Grade	Meaning
A	Distinction
B	Very Good
C	Credit
S	Pass
W	Fail
AB	Absent

Invalid values shall be rejected.

16. Overall Result Status

Possible statuses:

- Passed
- Not Passed
- Absent
- Incomplete

The exact business rules shall be configurable according to examination organizers' requirements.

Example rules:

Passed

No W grades in subjects defined as required.

Not Passed

One or more W grades in required subjects.

Absent

Student is absent according to the configured examination rule.

Incomplete

One or more required results are missing.

The calculation shall occur server-side.

17. Result Summary

The result page may display:

- Number of subjects.
- Passed subjects.
- Failed subjects.
- Absent subjects.
- Overall status.

It shall not display:

- Total marks.

- Average marks.
- Rank.

18. Result Actions

Students shall be able to:

- Print Result.
- Save/Download as PDF.
- Return to Search.

Optional future features:

- Result verification code.
- QR verification.
- Digitally verifiable result document.

19. Examination Management

The database shall support multiple examinations.

Examples:

```
PrepX 2026
PrepX 2027
PrepX 2028
```

An examination contains:

- Name.
- Year.
- Organization.
- Status.
- Publication date.
- Result notice.
- Created timestamp.
- Updated timestamp.

Suggested statuses:

```
DRAFT
READY
PUBLISHED
ARCHIVED
```

This allows future examinations without creating a new application or redesigning the database.

20. Publication Workflow

Results shall not be published student-by-student.

Publication shall occur at examination level.

Workflow:

```
DRAFT
|
v
VALIDATION
|
v
READY
```

```
|  
Admin Confirmation  
|  
v  
PUBLISHED
```

Before publication the system shall validate:

- Required student records.
- Unique Index Numbers.
- Unique NIC Numbers where required.
- Required subjects.
- Grade validity.
- Missing grades.
- Duplicate results.
- Incomplete records.

The administrator shall see a publication summary before confirmation.

Example:

```
Students: 200  
Completed Results: 198  
Incomplete Results: 2  
Invalid Grades: 0  
  
Warning:  
2 results are incomplete.  
  
Continue with publication?
```

21. Student Management

The administrator shall be able to:

- Add student.
- Edit student.
- Delete student where permitted.
- Search students.
- Filter by school.
- Filter by examination.
- Filter by result status.

Student fields:

- ID.
- Examination ID.
- Full name.
- Index Number.
- NIC Number.
- School.
- Examination center.
- Created timestamp.
- Updated timestamp.

Index Number uniqueness shall be enforced within the appropriate examination scope.

22. Subject Management

The administrator shall be able to:

- Add subjects.
- Edit subjects.
- Activate/deactivate subjects.
- Change display order.

Subject fields:

- ID.
- Examination ID.
- Subject name.
- Subject code.
- Display order.
- Required status.
- Active status.
- Created timestamp.
- Updated timestamp.

23. Grade Management

The administrator shall:

- Enter grades manually.
- Edit grades.
- Remove incorrect grades.
- Mark absence.
- Identify incomplete records.
- Save results before publication.
- Validate results.

Grade entry should use a controlled dropdown:

A
B
C
S
W
AB

The system shall reject unsupported values.

24. Excel / CSV Import

Supported formats:

.xlsx
.csv

Example:

Index	NIC	Name	School	Tamil	English	Mathematics
OL001	200312345678	Student A	School A	A	B	A

Upload workflow:

```

Upload
|
v
Parse File
|
v
Validate
|
v
Preview
|
v
Show Errors
|
v
Administrator Confirms
|
v
Database Transaction
|
+---- Success --> COMMIT
|
+---- Failure --> ROLLBACK

```

Validation shall detect:

- Missing columns.
- Invalid Index Numbers.
- Invalid NICs.
- Duplicate Index Numbers.
- Duplicate NICs.
- Invalid grades.
- Missing student names.
- Missing subjects.
- Duplicate student/subject results.

No import should unexpectedly leave partially written data.

25. Database Architecture

Supabase PostgreSQL shall be used.

Primary tables:

```

examinations
students
subjects
student_results
admins / authentication data
audit_logs

```

26. Examinations Table

Suggested fields:

```

id
name
year
organization_name

```

```
status
publication_date
result_notice
created_at
updated_at
```

27. Students Table

Suggested fields:

```
id
examination_id
full_name
index_number
nic_number
school_name
examination_center
created_at
updated_at
```

Important indexes:

```
(examination_id, index_number)
(examination_id, nic_number)
```

Appropriate unique constraints shall prevent duplicate identifiers within an examination.

28. Subjects Table

Suggested fields:

```
id
examination_id
subject_name
subject_code
display_order
required
active
created_at
updated_at
```

29. Student Results Table

Suggested fields:

```
id
student_id
subject_id
grade
created_at
updated_at
```

Required database constraint:

```
UNIQUE(student_id, subject_id)
```

This prevents duplicate subject results for a student.

Grade values shall also be validated at the database/application level.

30. Audit Logs

Important administrator operations shall create audit records.

Suggested fields:

```
id
admin_id
action
entity_type
entity_id
old_value
new_value
created_at
```

Important actions include:

```
STUDENT_CREATED
STUDENT_UPDATED
STUDENT_DELETED

RESULT_CREATED
RESULT_UPDATED
RESULT_DELETED

IMPORT_COMPLETED

EXAMINATION_PUBLISHED
EXAMINATION_UNPUBLISHED
```

Audit logs should not store secrets or unnecessary sensitive information.

31. Administrator Authentication

The administrator system shall use secure authentication.

Requirements:

- Passwords must never be stored as plaintext.
- Passwords must be securely hashed if custom password storage is used.
- Authentication shall happen server-side.
- Sessions shall expire.
- Admin routes shall require authentication.
- Logout shall invalidate the session.
- Login attempts shall be rate limited.
- Cookies shall use appropriate HttpOnly, Secure, and SameSite settings.
- Credentials shall never be committed to GitHub.

MFA should be enabled for the administrator where practical.

32. Supabase Security

The Supabase database shall not be exposed through unrestricted public database permissions.

Security shall include:

- Row Level Security where applicable.
- Minimal database permissions.
- Server-side privileged operations.
- Environment variables for secret credentials.
- No service-role secret in frontend code.
- No database password in client bundles.
- Input validation.
- Parameterized/database-library queries.

The frontend shall never receive unrestricted database access.

33. API Architecture

Suggested endpoints/actions:

```
POST /api/results/search

POST /api/admin/login
```

```
POST /api/admin/logout

GET /api/admin/students
POST /api/admin/students
PUT /api/admin/students/:id
DELETE /api/admin/students/:id

POST /api/admin/import

POST /api/admin/examinations/:id/publish
POST /api/admin/examinations/:id/unpublish

GET /api/health
```

Actual implementation may use Next.js Server Actions where more appropriate.

34. Data Privacy

Collect only information required for examination administration.

Required:

- Student name.
- Index Number.
- NIC Number where required.
- School.
- Examination information.
- Subject grades.

Avoid unnecessary collection of:

- Home address.
- Parent information.
- Phone number.
- Photographs.
- Unnecessary demographic information.

NIC Numbers shall be treated as sensitive identifiers.

They shall:

- Never be displayed completely on public pages.
- Never be included in public URLs.
- Not be unnecessarily logged.
- Be returned only in masked form.

35. Security Requirements

The application shall implement:

- HTTPS.
- Secure authentication.
- Authorization.
- Password hashing.
- Session security.
- Input validation.

- Rate limiting.
- Anti-enumeration protection.
- Secure cookies.
- Database constraints.
- Restricted Supabase access.
- Environment-variable secrets.
- Protection against SQL injection.
- Appropriate XSS protection.
- CSRF protections where relevant.
- Security headers.
- Safe error handling.

The public system must never expose:

- Database credentials.
- Environment variables.
- Service-role keys.
- Admin passwords.
- Complete NIC Numbers.
- Other students' results.
- Stack traces or internal server information.

36. Backup and Recovery

Database backup procedures shall exist before production publication.

Backups/recoverable exports should be created before:

- Major bulk imports.
- Major result modifications.
- Result publication where appropriate.

Recovery procedures shall be documented.

The administrator/developer should be able to recover from:

- Incorrect bulk import.
- Accidental deletion.
- Application errors.
- Database corruption or service failure.
- Incorrect result updates.

The backup capability available under the selected Supabase plan shall be reviewed before production deployment.

37. Monitoring and Observability

The application should monitor:

- Request volume.
- Result-search latency.

- API errors.
- HTTP 5xx responses.
- Authentication failures.
- Rate-limit events.
- Database errors.
- Slow queries.
- Deployment failures.

Logs shall not contain complete NIC Numbers, passwords, tokens, or other secrets.

A simple health endpoint shall be available:

```
GET /api/health
```

The endpoint shall not expose sensitive infrastructure details.

38. CI/CD and Deployment

Development workflow:

```
Developer
|
v
Git
|
v
GitHub
|
v
Automated Checks
|
+-- Lint
+-- Type Check
+-- Tests
+-- Build
|
v
Vercel Preview
|
v
Production Deployment
```

Separate environment configurations shall be maintained for development and production.

Production secrets shall be stored using secure environment-variable management.

39. Testing Requirements

Testing shall include:

Unit Testing

Test:

- Grade validation.
- Overall result calculations.
- NIC validation.
- Index Number normalization.
- Publication rules.

Integration Testing

Test:

```
API -> Supabase
Authentication -> Admin routes
Import -> Database
Publication -> Public result lookup
```

Security Testing

Test:

- Unauthorized admin access.
- Invalid sessions.
- Brute-force attempts.
- Result enumeration.
- Invalid input.
- SQL injection attempts.
- Access to unpublished results.

Load Testing

Use a load-testing tool such as k6.

Test:

```
100
250
500
750
1000
```

concurrent simulated users.

The production launch shall at minimum demonstrate acceptable behavior at the required 500-user load target.

40. User Interface Requirements

The application shall be:

- Mobile-first.
- Responsive.
- Lightweight.
- Professional.
- Accessible.
- Easy for students and parents.
- Usable on lower-end smartphones.
- Suitable for slower mobile connections.

Avoid:

- Large background videos.
- Excessive animations.
- Large JavaScript dependencies.
- Uncompressed images.
- Unnecessary network requests.

41. Recommended Pages

Public

```
/
/results
/results/not-found
/results/not-published
```

Administrator

```
/admin/login
/admin/dashboard
/admin/examinations
/admin/students
/admin/subjects
/admin/results
/admin/import
/admin/review
/admin/publication
/admin/audit-logs
```

42. Student User Flow

```
Open Website
|
v
Enter Verification Information
|
v
Submit
|
v
Client Validation
|
v
Server Validation
|
v
Rate Limit Check
|
v
Publication Check
|
v
Database Lookup
|
+---- Not Found --> Generic Error
|
+---- Found
|
v
Retrieve Grades
|
v
Sanitize Response
|
v
Display Result
|
+-----+-----+
| |
Print PDF
```

43. Administrator User Flow

```
Admin Login
|
v
Authentication
|
v
Dashboard
|
v
Select Examination
|
```

```

+-----+
| |
Manual Entry Excel Import
| |
+-----+-----+
|
v
Validation
|
v
Result Review
|
v
Publication Check
|
v
Admin Confirmation
|
v
Publish
|
v
Results Available

```

44. Result Example

UGSM O/L Model Examination Results 2026

Student Name: Mohamed Akeel
Index Number: OL2026001
NIC Number: *****5678
School: Example College

Subject Grade

Tamil A
English B
Mathematics B
Science C
History A
Religion A
ICT B
Commerce C

Overall Status: PASSED

[Print Result] [Download PDF]

45. Initial Production Infrastructure

The first production version shall use:

Frontend
Next.js + React + Tailwind CSS

Backend
Next.js server-side APIs/actions

Hosting
Vercel

Database
Supabase PostgreSQL

Security
HTTPS
Rate Limiting
Server-side validation
Secure admin authentication
Supabase security/RLS

Deployment

GitHub -> Vercel

Testing

Automated tests + k6 load testing

An external Redis cache is not required for the initial version.

Because the initial dataset is small and result lookups use indexed PostgreSQL queries, unnecessary infrastructure should be avoided.

Caching may be introduced later if real load testing demonstrates that it is necessary.

46. Components Intentionally Not Required Initially

The following technologies are deliberately excluded from the initial architecture:

- Kubernetes.
- Kafka.
- Microservices.
- Database sharding.
- Multiple database replicas.
- Multi-region deployment.
- Service mesh.
- Dedicated API gateway.
- Dedicated load-balancer infrastructure.
- Complex distributed caching.

These technologies may be considered only if future requirements justify them.

47. Pre-Launch Checklist

Before publishing real results:

- Production database configured.
- Database indexes created.
- Database constraints verified.
- Supabase security policies verified.
- Production environment variables configured.
- Admin password secured.
- MFA configured where available.
- HTTPS confirmed.
- NIC masking confirmed.
- Rate limiting tested.
- Unpublished-result protection tested.
- Excel import tested.
- Database recovery/export procedure tested.
- Audit logging tested.
- Mobile responsiveness tested.
- 500-user load test completed.

- 750/1000-user stress tests attempted.
- Production monitoring enabled.
- Error handling verified.
- Publication/unpublication tested.
- Final result data reviewed.

48. Future Enhancements

Possible future versions may add:

- Multiple administrator accounts.
- Role-based access control.
- Multiple organizations.
- QR result verification.
- Verification codes.
- SMS notifications.
- Email notifications.
- WhatsApp notifications.
- Advanced analytics.
- Historical examination portal.
- Advanced monitoring.
- Dedicated cache.
- Higher-capacity hosting/database infrastructure.

49. Final Architecture

```

SRI LANKAN USERS
|
HTTPS
|
v
VERCEL EDGE/CDN
|
+-----+-----+
| |
v v
Static Assets Next.js App
|
API / Actions
|
Rate Limiting
|
Validation
|
v
SUPABASE POSTGRESQL
|
+-----+-----+
| | |
Students Results Exams
| | |
+-----+-----+
|
Audit Logs

ADMINISTRATOR
|
v

```

```

Secure Login
|
v
Admin Panel
|
+-----+-----+
| | |
Manage Import Publish
Results Excel Results
| | |
+-----+-----+
|
v
Supabase PostgreSQL

```

The architecture is intentionally designed to be simple, secure, scalable, maintainable, and appropriate for the actual expected workload, rather than introducing unnecessary distributed-system complexity.